SISTEMA DE CONTROLE DE ACESSO

COFRE DE TOXINAS — MINISTÉRIO DO MEIO AMBIENTE

Documentação técnica completa do projeto

Arquitetura · Bibliotecas · Análise do código · Front-end · Fluxo de aprovação e reprovação · Níveis de acesso · Auditoria

Reconhecimento facial com DeepFace (VGG-Face), autenticação biométrica anti-fraude, três níveis cumulativos de acesso e trilha de auditoria permanente.

Disciplina: Engenharia de Software / Visão Computacional

Linguagem: Python 3.12 | Interface: Streamlit | Banco: SQLite3

Data de emissão: 07 de outubro de 2026

Sumário

1. Introdução
2. Arquitetura do sistema e estrutura do projeto
3. Importações e bibliotecas utilizadas
4. Análise do código-fonte — função de cada trecho
5. Front-end: o que foi usado e como foi estruturado
6. Banco de dados e modelo de dados
7. Níveis de acesso, permissões e avanços por nível
8. Fluxo geral: entrada de dados, aceite/rejeição da imagem, aprovação e reprovação
9. Registro de auditoria e trilha de auditoria
10. Segurança da informação: resumo das proteções
11. Conclusão

Referências

1. Introdução

1.1 Contexto e propósito

O projeto “Cofre de Toxinas — Ministério do Meio Ambiente” é uma aplicação web de controle de acesso a um cofre de segurança máxima, no qual são guardadas as informações mais sensíveis sobre o uso de toxinas no país. O sistema atua como a última linha de defesa contra tentativas de obtenção indevida desses dados: apenas pessoas devidamente autorizadas conseguem visualizar os compartimentos do cofre, e cada visualização é condicionada a um rigoroso esquema de permissões dividido em três níveis de segurança.

A autenticação é feita por reconhecimento facial (biometria) e/ou por usuário e senha, com verificação de “vivacidade” (liveness) para impedir que fotografias, impressões ou vídeos gravados sejam apresentados à câmera. Toda tentativa de acesso — aprovada ou reprovada — é registrada em uma trilha de auditoria permanente, incluindo as reprovações e as fraudes detectadas.

1.2 Objetivos

Objetivo geral:

Desenvolver um sistema de controle de acesso biométrico que garanta, de forma rastreável, que somente colaboradores autorizados visualizem as informações do cofre de toxinas, respeitando níveis hierárquicos cumulativos e registrando toda a atividade para fins de auditoria.

Objetivos específicos:

- Autenticar colaboradores por reconhecimento facial (DeepFace / VGG-Face com distância de cosseno) e por credenciais (usuário/senha com hash PBKDF2-HMAC-SHA256 e salt).
- Validar a autenticidade da imagem capturada por meio de um mini-vídeo anti-fraude com seis barreiras de liveness (janela temporal, movimento, padrão orgânico, variação 3D, piscada e detecção de tela).
- Aplicar permissões por níveis cumulativos (1, 2 e 3), liberando parcels progressivas do cofre conforme a hierarquia do colaborador.
- Registrar permanentemente acessos, negativas, fraudes e erros (trilha de auditoria) e disponibilizar dashboards e relatórios exportáveis (CSV e PDF).
- Registrar toda tentativa de acesso reprovada na trilha de auditoria, permitindo a análise posterior das falhas de segurança.

- Bloquear temporariamente o compartimento quando houver tentativas repetidas de acesso indevido (3 ocorrências em 10 minutos geram bloqueio de 15 minutos).
- Oferecer interface web responsiva, construída com Streamlit, com home gráfica, máscaras clicáveis e painéis analíticos.

1.3 Público-alvo e escopo

O sistema foi dimensionado para o Ministério do Meio Ambiente, com três perfis de usuário: permissão geral (nível 1), diretores de divisões específicas (nível 2) e Ministro do Meio Ambiente (nível 3). O escopo inclui cadastro de colaboradores com foto, validação biométrica de acesso, painel do cofre com três compartimentos, dashboards de auditoria, gestão de credenciais e relatórios. Não faz parte do escopo a integração com sistemas externos de RH ou a publicação em ambiente de nuvem.

2. Arquitetura do sistema e estrutura do projeto

2.1 Arquitetura lógica

A aplicação é um monolito web em Python organizado em três camadas lógicas, todas hospedadas no mesmo processo:

Camada	Responsabilidade	Onde está no código
Apresentação (front-end)	Telas, componentes, máscaras HTML, CSS, navegação, gráficos	app.py — funções _render_*, _aba_*, sidebar e CSS (linhas 42–153, 2982–5006)
Negócio	Autenticação, liveness, decisão de conceder/negar, bloqueio, relatórios	app.py — funções de validação e serviço (linhas 289–2925)
Dados	Persistência em SQLite, migrações, arquivos de fotos e imagens	controle_acesso.db, fotos_cadastradas/, IMAGEMHOME/ (linhas 148–286)

O servidor é o próprio Streamlit, executado com “streamlit run app.py”, escutando em 0.0.0.0:8501 (configurado em .streamlit/config.toml). Cada interação do usuário reexecuta o script, e o estado é mantido em st.session_state.

2.2 Estrutura de pastas e arquivos

Arquivo / pasta	O que é e para que serve
app.py	Código principal da aplicação: front-end, regra de negócio, banco, biometria e relatórios.
controle_acesso.db	Banco de dados SQLite (colaboradores, logs, tentativas, configurações). Criado automaticamente na primeira execução; não é versionado.
fotos_cadastradas/	Diretório local com as fotos biométricas dos colaboradores (base de comparação do reconhecimento facial).
IMAGEMHOME/	Imagem de capa usada na tela inicial (home); a primeira imagem encontrada é exibida, com máscaras clicáveis posicionadas sobre ela.
gerar_excel.py	Script auxiliar que gera o relatório executivo Relatorio_Controle_Acesso_Facial.xlsx com openpyxl (formatação corporativa).
requirements.txt	Dependências Python do projeto (versões

	restritas).
.streamlit/config.toml	Configuração do servidor: porta 8501, bind 0.0.0.0, maxMessageSize 200 MB (necessário para uploads de imagem).
README.md	Leia-me do projeto: tecnologias, níveis, credenciais e execução.
documentacao_cofre_toxinas.*	Documentação anterior do sistema (DOCX/HTML/PDF).
.gitignore	Exclui do versionamento o banco, venv/, .waprofile/ e relatórios gerados.
venv/	Ambiente virtual Python 3.12 com todas as dependências.

2.3 Como executar

```
python -m venv venv
source venv/bin/activate
pip install -r requirements.txt
streamlit run app.py
# relatório Excel auxiliar:
python gerar_excel.py
```

Requisitos: Python 3.12.

3. Importações e bibliotecas utilizadas

As importações de app.py estão concentradas nas linhas 1 a 33, com importações pontuais dentro de funções (re na linha 479, vl_convert na linha 766 e dlib na linha 1353). A tabela a seguir descreve, para cada biblioteca: o que é, para que serve neste projeto e onde é utilizada.

3.1 Biblioteca padrão do Python

Biblioteca (import)	Linha	O que é / função no projeto
os	1	Interação com o sistema de arquivos: monta caminhos de fotos e imagens, verifica existência de arquivos (os.path.exists) e remove imagens temporárias.
io	6	Streams em memória: converte bytes de imagem/PNG em objeto de imagem PIL (io.BytesIO) para miniaturas e gráficos.
hashlib	7	Primitivas criptográficas: PBKDF2-HMAC-SHA256 com 100.000 iterações para hash de senhas (linha 289) e SHA-1 para detectar quadros repetidos do mini-vídeo (linha 1882).
base64	8	Codifica a imagem da home em Base64 para ser embutida no HTML das máscaras clicáveis (linha 2996).
glob	9	Busca localização do preditor facial do dlib em site-packages (linha 1358).
json	10	Serializa os detalhes/dicionários dos logs (motivo da negativa, métricas de fraude) em colunas TEXT do SQLite (linhas 1772 e 2019).
secrets	11	Gera salt criptográfico (token_hex) e compara senhas de forma que não vaza tempo de execução (compare_digest) —

		linhas 291 e 307.
time	12	Carimbo de tempo da sessão (login_ts) e controle da janela de inatividade de 5 minutos.
datetime, timedelta	13	Data/hora dos logs, janela de fraude (10 min), duração do bloqueio (15 min) e datas de cadastro/validade.
sqlite3	16	Banco de dados relacional embutido: cria tabelas, migrações, inserções de logs e consultas de auditoria.
math	31	Funções trigonométricas usadas na geração da série temporal simulada de movimentação (linha 682).
re	479	Expressões regulares para normalizar rótulos legados de compartimento e extrair o nível a partir do texto do log.
site	1354	Obtém os diretórios de pacotes instalados para localizar o shape predictor do dlib.

3.2 Bibliotecas de terceiros

Biblioteca (import)	Linha	O que é / função no projeto
streamlit	29	Framework de interface web em Python. É o front-end inteiro da aplicação: componentes de formulário, câmera, tabelas, gráficos, navegação lateral, estado de sessão e cache. st.set_page_config (linha 35) define título, ícone e layout wide; st.cache_resource (linha 2918) mantém o modelo facial carregado uma única vez.

deepface	20	Biblioteca de análise facial de ponta. DeepFace.verify comparando duas imagens com model_name='VGG-Face', detector_backend='skip' e distance_metric='cosine' é o coração do reconhecimento (linha 1653); DeepFace.build_model carrega o modelo (linha 2922).
cv2 (opencv-python-headless)	18	Visão computacional: decodifica quadros (imdecode), converte cores, redimensiona, calcula diferença absoluta entre quadros (movimento), componentes conexos (padrão orgânico), recorta o centro do rosto e processa a FFT na detecção de tela/Moiré.
numpy	19	Estruturas numéricas multidimensionais: matrizes de pixels, pilha de regiões de variação, médias, percentis e variância.
PIL (Pillow)	17	Manipulação de imagens: abre, redimensiona e salva fotos; gera miniaturas do mini-vídeo; abre PNGs dos gráficos para o PDF.
pandas	21	Tabelas em memória: carrega os logs do SQLite (read_sql_query), agrupa métricas dos dashboards, normaliza datas e alimenta os gráficos Altair e os relatórios.
altair	30	Biblioteca de visualização declarativa (Vega-Lite). Gera todos os gráficos dos dashboards e do painel do cofre (barras, linhas, círculos, área) com tooltips interativos.
vl_convert	766	Renderiza a especificação Vega-Lite do Altair em PNG em memória, para incorporar os gráficos ao relatório PDF (import

		local).
fpdf (fpdf2)	33	Geração do relatório PDF: classe _RelatorioPDF (linha 774) com rodapé, fontes DejaVu (acentuação), seções, KPIs, gráficos e tabelas — função gerar_relatorio_pdf (linha 909).
dlib	1353	Preditor de 68 landmarks faciais usado para medir a abertura dos olhos (EAR) e detectar a piscada obrigatória do anti-fraude (import local, com cache em _carregar_preditor_dlib).
tf_keras / tensorflow	(dependências)	Back-end de deep learning do DeepFace/RetinaFace (declarados em requirements.txt, usados indiretamente pelo DeepFace).
retina-face	(dependência)	Detector de rostos alternativo disponível ao DeepFace (declarado em requirements.txt).
openpyxl	gerar_excel.py	Escrita do relatório executivo em .xlsx com estilos (fontes, cores, bordas) — não é importado por app.py.
geopandas / shapely	(declaradas)	Dependências geoespaciais declaradas em requirements.txt; não são importadas pelo código atual de app.py (reserva para expansão).

3.3 Observações sobre as dependências

- requirements.txt fixa faixas de versão (por exemplo: streamlit >=1.62, deepface >=0.0.100, pandas >=3.0.5, altair >=6.2.2, opencv-python-headless >=5.0) para garantir reprodutibilidade.
- vl_convert é dependência transitória do Altair e é importado localmente dentro de _chart_png (linha 766) apenas quando um PDF é gerado.
- A instalação de deepface/tensorflow/dlib é a etapa mais pesada do ambiente; o modelo VGG-Face é baixado no primeiro uso e mantido em memória por st.cache_resource.

4. Análise do código-fonte — função de cada trecho

A seguir, o app.py é decomposto por blocos funcionais. As linhas indicadas referem-se ao arquivo app.py do repositório.

4.1 Configuração inicial e identidade visual (linhas 35–153)

- st.set_page_config (35): título “Cofre de Toxinas — Ministério do Meio Ambiente”, ícone 🔐 e layout wide (tela larga).
- MOBILE_CSS (42–78): CSS injetado via st.markdown que torna a interface responsiva em telas de até 768 px (colunas empilhadas, sidebar com largura máxima, botões em largura total).
- MOLDURA_CIRCULAR_CSS (81): desenha a moldura circular sobre o widget de câmera, orientando o usuário a centralizar o rosto.
- HOME_CSS (108): estilos da tela inicial (home) com imagem de capa.
- IMAGEMHOME_DIR/EXTENSOES_IMAGEM (105–106): pasta e extensões aceitas para a imagem da home.
- DB_FILE e FOTOS_DIR (148–149): nomes do banco SQLite e da pasta de fotos.

4.2 Banco de dados e migrações (linhas 155–286)

- init_db (155): cria, se não existirem, as cinco tabelas — colaboradores, logs_acesso, logs_erros, tentativas_segurança e configuracoes.
- migrar_banco_legado (219): migra bancos antigos para o novo esquema usando PRAGMA user_version (v1→v4): adiciona as colunas nivel_acesso, divisao, detalhes, usuario, senha_hash e pode_cadastrar; zera níveis antigos para 1 na primeira migração; cria índice único para o usuário de login.
- Na sequência (407–408) init_db() e migrar_banco_legado() rodam na inicialização do script.

4.3 Autenticação por credenciais (linhas 289–343 e 2931–2980)

- hash_senha (289): gera salt aleatório de 16 bytes e calcula PBKDF2-HMAC-SHA256 com 100.000 iterações; armazena no formato “salt$hash”.
- verificar_senha (298): recomputa o hash a partir do salt armazenado e compara com secrets.compare_digest (comparação em tempo constante).
- _autenticar (2931): consulta o usuário no banco (COLLATE NOCASE), valida a senha, inicia a sessão e devolve (ok, mensagem).
- _render_criar_admin (3350): cria o primeiro administrador (nível 3) quando não existe nenhuma credencial no sistema.

4.4 Sessão, permissões e utilitários de domínio (linhas 410–763)

- NIVEIS_ACESSO (410): dicionário com nome e descrição dos três níveis — Permissão Geral, Diretor de Divisões Específicas e Ministro do Meio Ambiente.
- nome_nivel / descricao_nivel / rotulo_nivel (438–455): rotulam os níveis em textos amigáveis (“Nível 2 – Diretor de Divisões Específicas”).
- COMPARTIMENTOS_LEGADOS e normalizar_compartimento (457–490): convertem rótulos antigos de setor para o compartimento atual do cofre, preservando a leitura histórica dos logs.
- DADOS_TOXINAS (501–654): base de domínio com 10 toxinas de exemplo (Ricina, Saxitoxina, Toxina Botulínica etc.), cada uma com classe, periculosidade, compartimento, divisão responsável, estoque, validade e estado de conservação.
- DIVISOES_MMA (655): seis divisões do Ministério usadas no cadastro (Gabinete do Ministro, Toxicologia Ambiental, Emergências Químicas, Fiscalização e Segurança Química, Pesquisa e Laboratório, Serviços Gerais).
- carregar_dados_toxinas / serie_temporal_movimentacao (665–690): preenchem os gráficos do cofre e a série temporal de movimentações.
- _sessao_valida (694): sessão válida somente se autenticada e dentro do limite de inatividade (LIMITE_INATIVIDADE_S = 5 minutos, linha 691); ao expirar, limpa o estado e sinaliza sessao_expirada.
- usuario_tem_nivel (710): aplica a regra cumulativa nivel_acesso >= nivel_requerido.
- usuario_pode_cadastrar (718): libera a tela de cadastro apenas para quem tem a permissão pode_cadastrar.
- _preparar_logs (742): normaliza os logs para análise (extrai nível, compartimento curto, data e hora) alimentando os dashboards.

4.5 Relatórios e exportação (linhas 764–1276)

- _chart_png (764): converte gráfico Altair em PNG em memória com vl_convert.
- _RelatorioPDF (774) e auxiliares (787–907): classe FPDF com rodapé numerado, registro da fonte DejaVu (acentuação), seções, KPIs em cartões, inserção de gráficos e tabelas estilizadas.
- gerar_relatorio_pdf (909): monta o PDF completo com capa, sumário executivo, KPIs, gráficos e tabelas de auditoria; respeita os filtros ativos quando chamada pelo Dashboard Analítico.

4.6 Reconhecimento facial e liveness (linhas 1278–1636)

- recortar_centro_rosto (1278): recorta a região central da imagem antes da comparação, padronizando o enquadramento.

- _detector_mtcnn (1296): carrega e cacheia o detector MTCNN do RetinaFace para achar o rosto e os keypoints nos quadros.
- _carregar_preditor_dlib (1348): localiza e cacheia o shape predictor de 68 landmarks do dlib.
- _ear_dos_olhos (1376): calcula o Eye Aspect Ratio dos dois olhos; _piscada_detectada (1404) identifica a queda da abertura abaixo de 60% do baseline (LIMIAR_PISCADA = 0.60).
- _variacao_paralaxe (1423): mede a variação relativa entre nariz, olhos e boca ao longo dos quadros (variação 3D).
- _detectar_tela (1447): aplica FFT na região do rosto e mede a energia de alta frequência (padrão de Moiré = tela exibida à câmera).
- avaliar_liveness (1477): função central do anti-fraude. Recebe os quadros do mini-vídeo e devolve (aprovado, motivo, detalhes, melhor quadro) aplicando as seis barreiras descritas na seção 8.3.

Limiares utilizados (linhas 1326–1331):

Constante	Valor	Significado
LIMIAR_MOVIMENTO	0,015 (1,5%)	Movimento mínimo médio entre quadros consecutivos.
LIMIAR_N_COMPOS	4	Mínimo de regiões de variação (padrão orgânico do rosto).
LIMIAR_PARALAXE	0,05	Variação geométrica 3D mínima entre nariz/olhos/boca.
LIMIAR_PISCADA	0,60	Queda da abertura do olho (EAR) que caracteriza a piscada.
JANELA_MINIMA_S	2,0 s	Janela mínima entre o primeiro e o último quadro.
LIMIAR_TELA_FFT	12,0	Energia de alta frequência acima da qual a imagem é tida como tela.

4.7 Decisão de acesso (linhas 1639–1749)

- _melhor_match_biometrico (1639): percorre todos os colaboradores cadastrados, compara a foto testante com cada foto armazenada via DeepFace.verify (VGG-Face, cosseno) e devolve o candidato de menor distância.
- _conceder_ou_negar (1672): decisão final. Se nivel_colab >= nivel_requerido, exibe banner de sucesso, registra log PERMITIDO, inicia a sessão (usuario_logado com id, nome, nível, usuário, pode_cadastrar, login_ts e método) e redireciona ao cofre. Caso

contrário: banner de erro, log NEGADO com o motivo “Nível insuficiente” e registro em tentativas_segurança.

4.8 Política de bloqueio temporário (linhas 1752–1857)

Constante / função	Valor / papel
JANELA_FRAUDE_S	600 s (10 minutos) — janela de contagem de tentativas.
DURACAO_BLOQUEIO_S	900 s (15 minutos) — duração do bloqueio.
MAX_TENTATIVAS	3 — negativas/fraudes que disparam o bloqueio.
_registrar_tentativa_fraude (1758)	Insere em tentativas_segurança e retorna o nº de ocorrências na janela.
_bloqueio_ativo (1790)	Retorna (bloqueado, minutos restantes) calculando o término a partir da última tentativa.
_tentativas_recentes (1824)	Lista as últimas tentativas do compartimento (exibidas na tela de acesso bloqueado).
_registrar_e_verificar_bloqueio (1841)	Grava a tentativa e verifica o bloqueio do compartimento.

4.9 Telas de validação biométrica (linhas 1859–2003 e 3445–3553)

- _render_mini_video (1859): interface do mini-vídeo anti-fraude — captura de 4 a 8 quadros com deduplicação por SHA-1, miniaturas, contador e botão de análise; ao reprovar, registra log e bloqueio; ao aprovar, chama a comparação biométrica com o melhor quadro.
- _aba_simulacao (3445): tela “Validar Acesso / Login Biométrico” — verifica bloqueio do compartimento, permite escolher entre foto única (webcam) ou mini-vídeo e executa a validação (linhas 3494–3553).

4.10 Auditoria, erros e utilitários (linhas 2006–2148)

- registrar_log (2006): insere em logs_acesso o colaborador, a área (compartimento), data/hora, status (PERMITIDO/NEGADO) e os detalhes em JSON.
- obter_ajuda_erro (2026): mapeia o tipo do erro (banco, biometria, imagem, geral) para uma explicação amigável e uma sugestão de solução — exibida na aba de logs.
- registrar_erro (2121) e limpar_logs_erros (2139): gravação e manutenção da tabela logs_erros.

4.11 Configurações persistentes (linhas 351–436)

- Funções de leitura e gravação de preferências na tabela configuracoes (chave, valor), usadas para persistir as posições das máscaras clicáveis da tela inicial e outras opções do sistema.
- As preferências são recarregadas a cada execução, garantindo que as configurações sobrevivam ao reinício da aplicação.

4.12 Cadastro de colaboradores (linhas 2830–2925)

- salvar_colaborador (2830): valida nome, foto, usuário e senha (mínimo de 6 caracteres), verifica duplicidade de usuário, grava a foto em fotos_cadastradas/ com recorte de rosto, gera o hash da senha e insere o registro com nível e divisão.
- carregar_modelo_facial (2918): @st.cache_resource carrega o VGG-Face uma única vez para todo o ciclo de vida do servidor.

4.13 Home, login e máscaras clicáveis (linhas 2982–3443)

- _encontrar_imagem_home / _imagem_home_base64 (2982–2997): localiza a primeira imagem da pasta IMAGEMHOME e a converte em Base64.
- _zonas_home_default / _carregar_zonas_home / _salvar_zonas_home (2999–3063): coordenadas (em %) das áreas clicáveis, com persistência no banco (chaves home_zona_*).
- _montar_html_mascaras (3064): gera o HTML/CSS com a imagem de fundo e as camadas transparentes posicionadas em porcentagem (escalam com a tela).
- _tratar_acao_home (3108): processa os cliques (query params acao, ajustar, ajuste_sair) e troca a tela da home.
- _render_home_fallback (3135): versão com botões textuais quando não há imagem em IMAGEMHOME.
- _render_ajuste_mascaras (3188): modo de ajuste com sliders X/Largura/Altura por zona e botão de salvar/restaurar.
- _render_home (3283) e _render_login (3312): montagem da home e do formulário de login (st.form) com redirecionamento inteligente (cadastro exige pode_cadastrar).
- _encerrar_sessao (3435): limpa o estado e encerra a sessão.

4.14 Navegação e rotas (linhas 3556–3798)

- st.sidebar (3556): título, indicador de sessão, aviso de sessão expirada e botão de encerrar sessão.

- _opcoes_nav + st.sidebar.radio (3595–3616): lista de rotas; a rota "🛡️ Gestão de Acessos" só aparece para o nível 3; a rota inicial pode ser alterada por st.session_state['redirecionar_para'].
- ao_trocar_aba_guard (3782): envolve cada rota em try/except, registrando qualquer erro inesperado em logs_erros e exibindo mensagem amigável.

4.15 Rotas (abas) do sistema (linhas 3799–5006)

Rota (sidebar)	Função no código	O que faz / restrição
🔐 **Cofre de Toxinas do MMA**	_aba_dados_geo (4776)	Painel com três compartimentos e governança; exige sessão ativa e aplica níveis cumulativos (1: catálogo; 2: inventário e gráficos; 3: dossiê, cadeia de custódia, KPIs e gestão).
📹 **Validar Acesso / Login Biométrico**	_aba_simulacao (3445)	Seleciona o nível do compartimento e valida por foto ou mini-vídeo anti-fraude.
📈 **Dashboard Analítico de Acessos**	_aba_dash_analitico (3946)	Tendências, distribuição por compartimento, equipe do cofre, diagnósticos de segurança e exportação de PDF; restrito a níveis 2 e 3.
📊 **Dashboard e Relatórios**	_aba_dash_relatorios (4321)	Métricas de permitidos/negados, filtros, exportação CSV e PDF completo; restrito a níveis 2 e 3.
🐞 **Registro de Erros / Logs**	_aba_logs (4448)	Lista de erros com explicação e solução, logs de acesso e limpeza; restrito a níveis 2 e 3.
📸 **Cadastro de Colaboradores**	_aba_cadastro (3800)	Formulário com foto (webcam ou upload), divisão, nível e credenciais; exige sessão + pode_cadastrar.
🛡️ **Gestão de Acessos**	_aba_gestao_acessos (4610)	Edita usuário, senha, nível e permissão de cadastro; restrito ao nível 3.

5. Front-end: o que foi usado e como foi estruturado

5.1 Tecnologia

Todo o front-end foi construído com Streamlit (Python), ou seja, não existe HTML/JavaScript/CSS separado: os componentes são declarados em Python e renderizados pelo servidor Streamlit no navegador. O CSS é injetado em três pontos específicos (MOBILE_CSS, MOLDURA_CIRCULAR_CSS e HOME_CSS) por meio de st.markdown(..., unsafe_allow_html=True), e a home usa HTML gerado dinamicamente para as máscaras clicáveis.

Aspecto	Implementação
Servidor	Streamlit embutido — 0.0.0.0:8501 (.streamlit/config.toml), maxMessageSize 200 MB.
Layout	layout='wide' (tela larga) com título e ícone personalizados.
Responsividade	MOBILE_CSS com media query @media (max-width: 768px): colunas empilhadas, sidebar com largura máxima, métricas e títulos menores.
Estado	st.session_state guarda sessão autenticada, usuario_logado, telas da home e quadros do mini-vídeo.
Cache	@st.cache_resource (modelo facial) e @st.cache_data (imagem da home em Base64).
Navegação	st.sidebar.radio com opções dinâmicas (Gestão de Acessos apenas para o nível 3).
Feedback	st.toast, st.success, st.warning, st.error, st.spinner e banners próprios (_render_banner_auth).

5.2 Estrutura das telas

A aplicação tem duas grandes fases de navegação:

1. Pré-login (sem sessão): a home com imagem de capa e máscaras clicáveis. O script verifica _sessao_valida() na linha 3558 e, se inválida, renderiza apenas a home/tela de entrada e interrompe a execução com st.stop() (linha 3590) — nenhum dado sensível é renderizado.
2. Pós-login (sessão ativa): sidebar com navegação, indicador de usuário e nível, botão de encerrar sessão e a rota selecionada, cada uma isolada em ao_trocar_aba_guard().

5.3 Home com máscaras clicáveis

A tela inicial exibe a imagem encontrada em IMAGEMHOME/ e sobre ela posiciona camadas transparentes (divs HTML) que funcionam como botões: Usuário e Senha, Acesso (biometria), Cadastro e Criar Primeiro Administrador (este último apenas quando não há credenciais). As coordenadas são armazenadas em porcentagem, portanto as áreas escalam junto com a imagem em qualquer tamanho de tela. No rodapé há o link "⚙️ Ajustar áreas clicáveis", que abre um modo de edição com bordas tracejadas, sliders X/Y/Largura/Altura e gravação das posições no banco (tabela configuracoes, chaves home_zona_*). Se a pasta estiver vazia, a home cai para uma versão com botões textuais (_render_home_fallback).

5.4 Componentes Streamlit utilizados

Componente	Uso no projeto
st.camera_input	Captura de foto (cadastro e validação) e dos quadros do mini-vídeo.
st.file_uploader	Upload de imagem JPG/PNG no cadastro.
st.form / st.form_submit_button	Login e edição de colaborador (agrupam campos e evitam reexecução).
st.selectbox / st.multiselect / st.radio / st.slider	Escolha de nível, divisão, origem da imagem, filtros de dashboard e ajuste das máscaras.
st.dataframe	Tabelas de colaboradores, logs, inventário, cadeia de custódia e tentativas.
st.altair_chart	Gráficos de barras, linhas, círculos e área (Altair/Vega-Lite).
st.metric	KPIs dos dashboards e do painel do cofre.
st.download_button	Exportação de CSV e PDF em várias telas.
st.columns / st.container / st.expander / st.divider	Layout em colunas, cartões, seções recolhíveis e separadores.
st.query_params	Ações da home (acao, ajustar) de forma reprodutível entre recargas.
st.rerun / st.stop / st.session_state	Controle de fluxo, redirecionamentos e estado.
st.sidebar	Título, navegação, sessão e configurações.

5.5 Identidade visual

Os estilos injetados definem moldura circular na câmera (orientando o usuário a centralizar o rosto), botões em largura total no mobile, títulos menores em telas pequenas e a estética

de cartões da home. As cores das mensagens seguem a semântica do Streamlit: verde = aprovado, vermelho = reprovado, azul/informativo = estado do sistema.

6. Banco de dados e modelo de dados

Banco SQLite3 em arquivo (controle_acesso.db), acessado diretamente com sqlite3. As tabelas são criadas por init_db() e evoluídas por migrar_banco_legado().

6.1 Tabelas

Tabela	Campos	Finalidade
colaboradores	id, nome, cargo, divisao, nivel_acesso (1–3), usuario, senha_hash (PBKDF2+salt), pode_cadastrar (0/1), foto_path, data_cadastro	Cadastro de pessoas, credenciais, nível e foto biométrica.
logs_acesso	id, colaborador_nome, setor (compartimento), data_hora, status (PERMITIDO/NEGADO), detalhes (JSON)	Trilha de auditoria de cada tentativa de acesso.
logs_erros	id, data_hora, tipo, modulo, mensagem, descricao, sugerida	Registro de erros com explicação amigável e solução sugerida.
tentativas_seguranca	id, compartimento, data_hora, motivo, detalhes, origem (foto/mini_video)	Contagem de fraudes e negativas que acionam o bloqueio temporário.
configuracoes	chave, valor	Preferências persistentes (posições das máscaras da home e outras opções do sistema).

6.2 Migrações (PRAGMA user_version)

Versão	Alteração
v1	Adiciona nivel_acesso e divisao; colaboradores legados são reiniciados para o nível 1.
v2	Adiciona a coluna detalhes em logs_acesso (motivo/métricas da negativa).
v3	Cria a tabela configuracoes.
v4	Adiciona usuario, senha_hash e pode_cadastrar; cria índice único sobre usuario.

Também é normalizado o campo divisao (‘Sem divisão’ quando vazio), garantindo consistência para os agrupamentos dos dashboards.

7. Níveis de acesso, permissões e avanços por nível

7.1 Os três níveis (cumulativos)

Nível	Nome	O que libera
1	Permissão Geral	Catálogo público de toxinas e informações gerais do cofre.
2	Diretor de Divisões Específicas	Tudo do nível 1 + inventário, localização, conservação, dashboards analítico/relatórios e aba de logs.
3	Ministro do Meio Ambiente	Tudo dos níveis 1 e 2 + dossiê completo, movimentações, cadeia de custódia, governança/auditoria e Gestão de Acessos.

Regra de concessão (linha 1690): acesso permitido quando nivel_colab >= nivel_requerido. Ou seja, um colaborador de nível N acessa todos os compartimentos dos níveis 1 a N — os níveis são acumulativos, nunca excludentes.

7.2 Matriz de permissões por tela

Tela	Nível 1	Nível 2	Nível 3	Observação
Cofre (catálogo/inventário/dossiê)	Catálogo	Inventário	Dossiê + governança	Blocos internos também verificam o nível.
Validação biométrica	Sim	Sim	Sim	Sessão autenticada.
Dashboard Analítico	Não	Sim	Sim	usuario_tem_nivel(2).
Dashboard e Relatórios	Não	Sim	Sim	usuario_tem_nivel(2).
Registro de Erros / Logs	Não	Sim	Sim	usuario_tem_nivel(2).
Cadastro de Colaboradores	Não	Não	Concede	Exige sessão + pode_cadastrar.
Gestão de Acessos	Não	Não	Sim	usuario_tem_nivel(3).

7.3 Avanço de acesso entre níveis

O avanço de nível é uma ação administrativa do nível 3 (Ministro), executada na aba 🛡️ Gestão de Acessos:

3. O gestor seleciona o colaborador e altera o nível de acesso, o usuário, a senha e/ou a permissão de cadastro.
4. Os dados são gravados na tabela colaboradores (a senha é re-hasheada com PBKDF2).
5. A partir da próxima autenticação, a sessão recebe o novo nivel_acesso e a regra cumulativa libera (ou restringe) os compartimentos imediatamente.
6. A permissão pode_cadastrar deve ser concedida a apenas 1 ou 2 colaboradores de confiança — o próprio sistema é quem decide, evitando autocadastro.

Colaboradores cadastrados antes do esquema de credenciais acessam pelo botão 🔐 Acesso da home (biometria) e podem receber usuário/senha depois, pelo nível 3.

8. Fluxo geral: entrada de dados, aceite/rejeição da imagem, aprovação e reprovação

8.1 Entrada de dados

Existem duas entradas de imagem no sistema:

- Cadastro (entrada de base biométrica): foto tirada pela webcam ou upload de JPG/PNG. É aceita somente se houver nome, foto e usuário informados, senha com no mínimo 6 caracteres e usuário não duplicado; a foto é salva em fotos_cadastradas/, recortada no centro do rosto e associada ao colaborador (salvar_colaborador, linha 2830).
- Validação de acesso (entrada de dados em tempo real): o usuário escolhe o nível do compartimento, e o sistema captura uma foto única pela webcam ou um mini-vídeo de 4 a 8 quadros. A imagem é então submetida ao pipeline de aceite descrito em 8.3 — ela só é aceita se passar em todas as barreiras e na comparação biométrica.

8.2 Pré-condições antes da captura

- Sessão: a rota de validação pode ser aberta da home (sem senha, apenas biometria) ou com sessão ativa.
- Bloqueio: antes de aceitar a imagem, _bloqueio_ativo() consulta tentativas_segurança; se houver 3 ocorrências em 10 minutos no compartimento, a validação fica suspensa por até 15 minutos e a tela exibe as últimas tentativas (retorna sem capturar nada).

8.3 O sistema aceita ou rejeita a imagem? (cadeia de decisões)

No modo mini-vídeo, avaliar_liveness() avalia os quadros nesta ordem; a primeira falha rejeita a imagem e interrompe o processo:

#	Verificação	Critério de aceitação	Se reprovar
1	Quantidade de quadros	Ao menos 3 quadros válidos (ideal 4 a 8)	Reprovado: captura insuficiente
2	Janela temporal	≥ 2,0 s entre o 1º e o último quadro (anti-rajada de fotos)	Reprovado: captura rápida demais
3	Detecção de rosto (MTCNN)	Rosto presente em ≥ 60% dos quadros (mín. 30×30 px)	Reprovado: imagem borrada/escura/fora do círculo
4	Movimento	Diferença média ≥ 1,5% entre quadros consecutivos	Reprovado: foto/tela parada

5	Padrão orgânico	≥ 4 regiões de variação (componentes conexos) — movimento não rígido	Reprovado: foto deslizando como peça única
6	Variação 3D (parallax)	≥ 0,05 na variação relativa entre nariz/olhos/boca	Reprovado: translação plana (foto impressa)
7	Piscada (EAR + 68 landmarks dlib)	Ao menos uma piscada (abertura do olho < 60% do baseline)	Reprovado: fotos e telas nunca piscam
8	Detecção de tela (Moiré/FFT)	Energia de alta frequência < 12,0	Reprovado: vídeo ou foto exibido em monitor/celular

Se todas as barreiras forem vencidas, o sistema seleciona o melhor quadro (maior rosto), recorta o centro e compara com todos os colaboradores cadastrados por DeepFace.verify (VGG-Face, distância de cosseno).

8.4 Aprovação e reprovação

Etapa	APROVADO ✓	REPROVADO ✗
Liveness	Motivo “Movimento 3D do rosto + piscada detectados” e métricas exibidas.	Motivo específico da barreira que falhou + dicionário de detalhes.
Biometria	Menor distância ≤ 0,35 → colaborador identificado (linhas 1967 e 3516).	Distância > 0,35 ou sem candidato → “Desconhecido”.
Nível	nivel_colab ≥ nível exigido → sessão iniciada e compartimentos liberados.	Nível insuficiente → negativa com o nível do colaborador vs. exigido.
Auditoria	log PERMITIDO em logs_acesso.	log NEGADO + gravação em tentativas_segurança.
Estado	usuario_logado preenchido; redireciona ao cofre; sessão expira em 5 min de inatividade.	Bloqueio temporário ao atingir 3 ocorrências em 10 min.
Interface	Banner verde “ACESSO AUTORIZADO!” com a distância e o compartimento.	Banner vermelho “ACESSO NEGADO!” com o motivo; mini-vídeo exibe 🚨 FRAUDE SUSPEITA.

8.5 Fluxo de aprovação e reprovação detalhado

7. Usuário abre a aplicação → home (pré-login) com máscaras clicáveis.
8. Escolhe LOGIN (usuário/senha → PBKDF2 → sessão) ou ACESSO (biometria, sem senha).
9. Na rota biométrica, seleciona o nível do compartimento (1, 2 ou 3).
10. O sistema checa o bloqueio temporário do compartimento.
11. Captura a imagem (foto única) ou os quadros do mini-vídeo.
12. Executa a cadeia de 8 verificações (seção 8.3).
13. Se qualquer barreira falhar → REPROVADO: log NEGADO, registro em tentativas_segurança e, ao chegar a 3 tentativas em 10 min, bloqueio de 15 min.
14. Se passar → comparação biométrica (VGG-Face/cosseno ≤ 0,35).
15. Se não reconhecer → REPROVADO (pessoa desconhecida) com as mesmas consequências.
16. Se reconhecer → verifica o nível do colaborador contra o nível exigido.
17. Nível suficiente → APROVADO: log PERMITIDO, sessão iniciada, compartimentos do nível (e inferiores) liberados.
18. Nível insuficiente → REPROVADO (nível insuficiente) com log e tentativa registrada.

8.6 Diagrama do fluxo geral

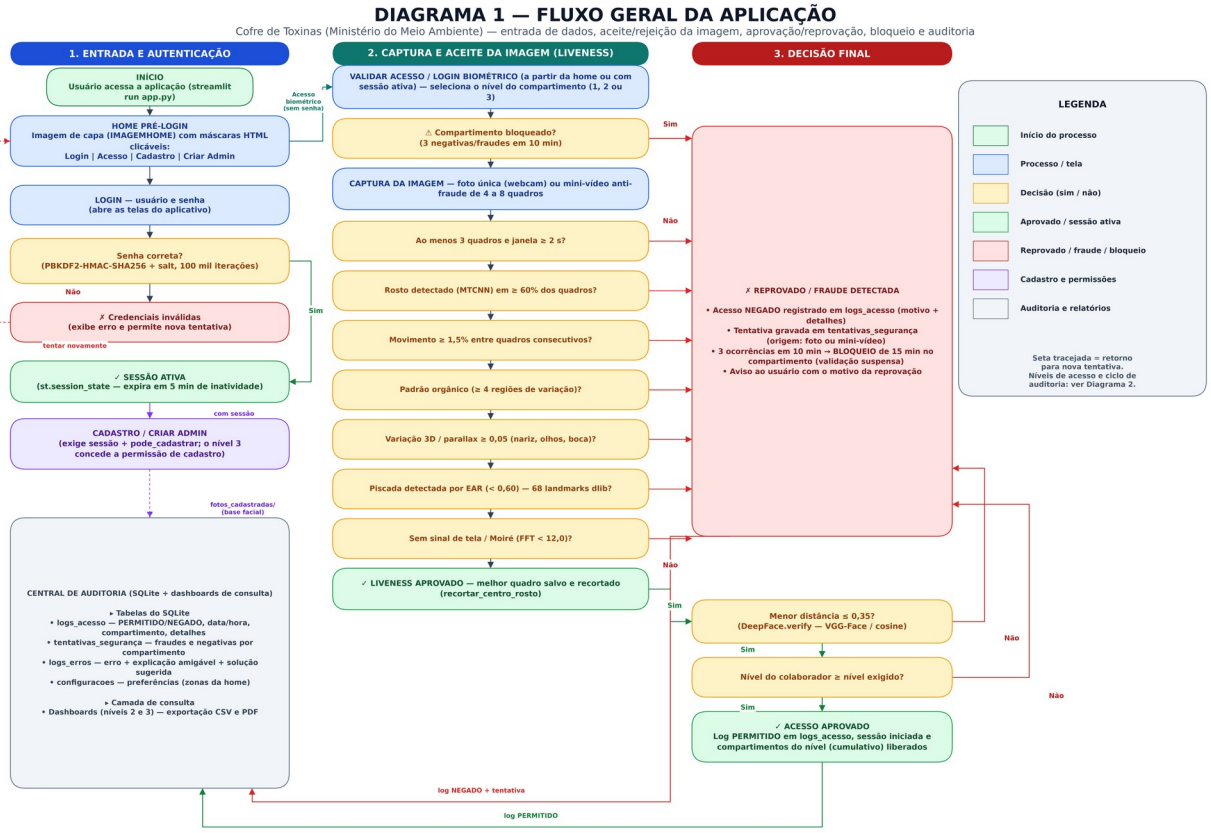


Diagrama 1 — Fluxo geral da aplicação: entrada e autenticação, captura/aceite da imagem (liveness), decisão final de aprovação/reprovação, bloqueio temporário e central de auditoria.

9. Registro e trilha de auditoria

9.1 O que é registrado

Registro	Quando é gravado	Campos-chave
logs_acesso (acesso)	Toda decisão de acesso (permitido ou negado), inclusive por nível insuficiente e biometria não reconhecida.	colaborador_nome, setor/compartimento, data_hora, status, detalhes (motivo, nível, métricas)
tentativas_segurança	Toda negativa ou fraude detectada (origem foto ou mini-vídeo), inclusive o bloqueio.	compartimento, data_hora, motivo, detalhes, origem
logs_erros (erros)	Qualquer exceção ou falha de operação (banco, biometria, imagem, geral).	data_hora, tipo, módulo, mensagem, descrição amigável, solução sugerida
configuracoes	Alteração das preferências e das zonas da home.	chave, valor

9.2 Quem consulta a auditoria

- 📊 Dashboard Analítico (níveis 2 e 3): tendências temporais, distribuição por compartimento, equipe do cofre, diagnósticos de segurança e exportação de PDF com os filtros ativos.
- 📈 Dashboard e Relatórios (níveis 2 e 3): permitidos vs. negados, filtros, CSV e PDF completo com KPIs, gráficos e tabelas.
- 🐞 Registro de Erros (níveis 2 e 3): erros com explicação e solução, além do histórico de acessos.
- 🏛️ Governança do cofre (nível 3): KPIs (toxinas, estoque, colaboradores, eventos de acesso) e tabela de colaboradores.

9.3 Resposta à reprovação

- Toda reprovação é gravada em tentativas_segurança com o motivo, a origem (foto ou mini-vídeo) e os detalhes da barreira que falhou.
- Ao atingir 3 ocorrências em 10 minutos, o compartimento é bloqueado por 15 minutos.
- A análise das reprovações e do histórico fica disponível nas telas de auditoria (Dashboard Analítico, Registro de Erros e Governança).

9.4 Diagrama de níveis e auditoria

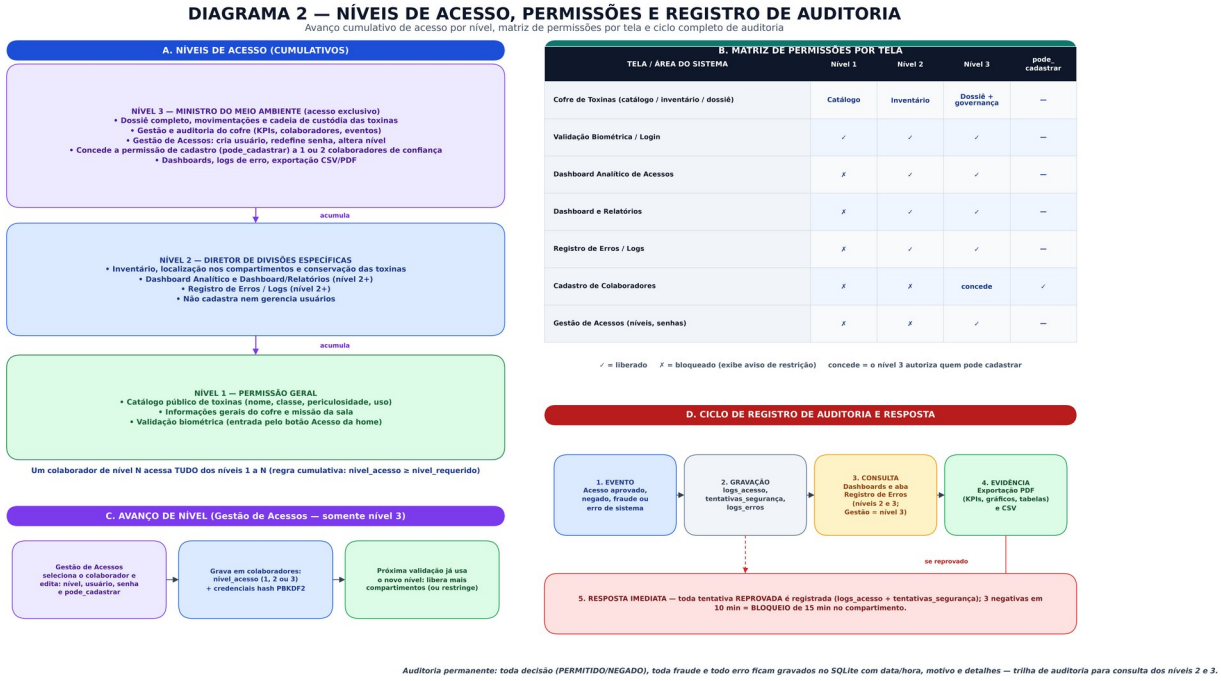


Diagrama 2 — Níveis de acesso cumulativos, avanço de nível, matriz de permissões por tela e ciclo de registro de auditoria e resposta.

10. Segurança da informação: resumo das proteções

Área	Proteção implementada	Onde
Senhas	PBKDF2-HMAC-SHA256 com salt único e 100.000 iterações; comparação em tempo constante; nunca armazenado texto puro.	hash_senha / verificar_senha (289–309)
Sessão	Sessão em st.session_state com expiração por inatividade de 5 minutos e aviso de sessão expirada.	_sessao_valida (694)
Autorização	Níveis cumulativos verificados em cada rota e em cada bloco; cadastro exige permissão extra.	usuario_tem_nivel / usuario_pode_cadastrar (710–723)
Anti-fraude	Seis barreiras de liveness no mini-vídeo (janela, movimento, padrão orgânico, parallax, piscada e Moiré).	avaliar_liveness (1477)
Bloqueio	3 negativas/fraudes em 10 min travam o compartimento por 15 min.	1752–1857
Auditoria	Logs de acesso, tentativas e erros com data/hora, motivo e detalhes.	registrar_log / registrar_erro
Primeiro acesso	Criação do primeiro administrador apenas quando não existe nenhuma credencial.	_render_criar_admin (3350)
Privacidade	Fotos e banco não são versionados (.gitignore); imagem temporária é removida após a validação.	.gitignore e linhas 1964/3513

11. Conclusão

O sistema Cofre de Toxinas demonstra, na prática, a aplicação de conceitos de visão computacional, segurança da informação e engenharia de software: reconhecimento facial com DeepFace (VGG-Face/cosseno), verificação de vivacidade com múltiplas barreiras, autenticação por hash de senha, controle de acesso hierárquico e cumulativo, política de bloqueio temporário e trilha de auditoria permanente.

O front-end, construído integralmente com Streamlit, oferece uma interface única e responsiva que reúne home gráfica com áreas clicáveis, painel do cofre, cadastro, validação biométrica, dashboards analíticos, gestão de acessos e central de erros — sempre respeitando as permissões do nível do usuário autenticado.

Como trabalhos futuros, podem-se considerar a substituição do SQLite por um banco cliente-servidor, a publicação da aplicação em ambiente web com HTTPS e MFA, a ampliação da base de dados de toxinas a partir de fontes oficiais e a inclusão de análise geoespacial (geopandas/shapely já declarados no requirements.txt).

Referências

CHOLLET, F. DeepFace: deep face recognition with Keras, TensorFlow and MTCNN. Documentação e repositório do projeto DeepFace.

PARKHI, O. M. et al. Deep Face Recognition. British Machine Vision Conference (BMVC), 2015 — arquitetura VGG-Face.

SCHROFF, F.; KALENCHENKO, D.; PHILBIN, J. FaceNet: A Unified Embedding for Face Recognition and Clustering — métrica de distância por cosseno.

KING, D. E. dlib: C++ Library with Python Bindings for Machine Learning — detector de landmarks faciais (68 pontos).

GUO, G. et al. How Far are We from a Practical Eye Aspect Ratio (EAR) Blink Detector.

STREAMLIT DOCUMENTATION. Building interactive web apps in Python.

PYTHON SOFTWARE FOUNDATION. hashlib — PBKDF2-HMAC, secrets and random number generation.

PROJETO Cofre de Toxinas — Ministério do Meio Ambiente. Repositório interno: app.py, README.md e requirements.txt.